COMP4141 Slides Week 3

Brendan Mabbutt

UNSW

March 2, 2026

Admin

Tests Moving Froward

Talk about the test.

- They are 20% in total (one quiz in week 2 – 5 and 7 – 10).
- They are Pass level questions
- Only one chance (the 20 minutes during the quiz)
- Based on **the previous week content.**
- Previous week's tutorial questions help with the assessment
- Best 7 of 8

Pumping Lemma & Myhill–Nerode Theorem

Pumping Lemma

Pumping Lemma

If $L \subseteq \Sigma^*$ is regular, then there exists $p \in \mathbb{N}$ (the *pumping length*) such that for every $w \in L$ with $|w| \geq p$, we can write $w = xyz$ satisfying:

1. $|xy| \leq p$,
2. $|y| > 0$,
3. $xy^iz \in L$ for all $i \in \mathbb{N}$.

$x \quad q \quad \boxed{y} \quad q \quad z$

$\hookrightarrow$ i, $xz \in L$

$xy^2z \in L$

same state (going around)

Myhill–Nerode Theorem

Distinguishability

Let $L \subseteq \Sigma^*$. $x, y \in \Sigma^*$

We say x and y are *distinguishable by* L if there exists $z \in \Sigma^*$ such that:

$$xz \in L \quad \text{and} \quad yz \notin L$$

(or vice versa).

We write $x \equiv_L y$ if x and y are not distinguishable by L.

The *index* of L is the number of $\equiv_L$-equivalence classes.

Myhill–Nerode Theorem

A language $L \subseteq \Sigma^*$ is regular if and only if the index of L is finite.

Moreover, this index equals the minimum number of states of any DFA that accepts L.

Using Myhill–Nerode

Myhill–Nerode proof format

Find an infinite sequence of words (not necessarily in L), $u_1, u_2, u_3, \ldots,$ and a doubly indexed sequence of words $w_{i,j}$ for $i < j \in \mathbb{N}_0$ such that

$$u_i w_{i,j} \in L \quad \text{and} \quad u_j w_{i,j} \notin L$$

(or vice versa).

$u_i w_{i,j} \notin L$ and $u_j w_{i,j} \in L$

Context Free Grammars

Formal Definition

Definition

A *context-free grammar* (*CFG*) is a 4-tuple (N, Σ, P, S), where

1. N is a finite set of *variables*,
2. Σ is a finite set, disjoint from N, of *terminals*,
3. $P \subseteq N \times (N \cup \Sigma)^*$ is a finite set of *rules*, and
4. $S \in N$ is the *start variable*.

Variables are often called *non-terminal symbols*, terminals are often called *terminal symbols*, rules also go under the name *productions*, and the start variable is also known as the *sentence symbol*.

Derivations

The language of a given CFG, $G = (N, \Sigma, P, S)$, can be characterized using the concept of a *derivation*.

Definition

Derivation step: $\alpha A \beta \Rightarrow_G \alpha \gamma \beta$ whenever $A \to \gamma \in P$.

Define $\Rightarrow_G^*$ to be the *reflexive transitive closure* of $\Rightarrow_G$. That is, $\alpha \Rightarrow_G^* \beta$ if we can get from α to β in zero or more steps.

The *language of G* is

$$L(G) = \left\{ w \in \Sigma^* \;\middle|\; S \Rightarrow_G^* w \right\}$$

Parse Tree

Definition

A **parse tree** is a tree that shows how to derive a string from a non-terminal.

Definition

The concatenation of the symbols at the leaves of a parse tree is called the **yield** of the parse tree.

Leftmost Derivations

Definition

A derivation of a string w in a grammar G is a **leftmost derivation** if at every step the leftmost remaining variable is the one replaced.

Definition

A context-free grammar (CFG) is **ambiguous** if there is more than one leftmost derivation for the same string.

Pushdown Automata

Definition

A **pushdown automaton (PDA)** is a 6-tuple $(Q, \Sigma, \Gamma, \delta, q_0, F)$ where:

1. Q is the set of states,
2. Σ is the input alphabet,
3. Γ is the stack alphabet,
4. $\delta : Q \times (\Sigma \cup \{\epsilon\}) \times (\Gamma \cup \{\epsilon\}) \to 2^{Q \times \Gamma^*}$ is the transition relation,
5. $q_0 \in Q$ is the start state, and
6. $F \subseteq Q$ is the set of accept states.

Acceptance by a PDA

Definition

An **instantaneous description (ID)** is a snapshot (q, w, α), where:

- $q \in Q$ is the current state,
- $w \in \Sigma^*$ is the portion of the input not yet read,
- $\alpha \in \Gamma^*$ is the complete contents of the stack.

A string w is **accepted** if there exists $\gamma \in \Gamma^*$ and $p \in Q$ such that

$$(q_0, w, \epsilon) \rightsquigarrow^* (p, \epsilon, \gamma), \quad p \in F.$$

Power of Pushdown Automata

Theorem

A language $L \subseteq \Sigma^*$ is **context-free** if and only if some PDA recognizes L.

W3 P1

Problem 1

Show that the following languages are not regular:

(a) $\{0^i 1^j : 0 \le i \le j\}$ $= \underbrace{000\ldots0}_{i}\underbrace{11\ldots1}_{j}$

L

more 1's then 0's

By contra.

Suppose L is regular, then $\exists\, p \in \mathbb{N}$ s.t.

①, ②, ③ apply

Want $|w| \geq p$ and $w \in L$

Try $0^p 1^p$

then by PL, $\exists\, x, y, z \in L^*$

- $|xy| \leq p$ then

$(\underbrace{00\ldots0}_{m})$ $xy = 0^m$ for $m \leq p$

- $|y| > 0$ so $\underline{y = 0^k}$ for some $k \leq p$

try: $xz = 0^{p-k} 1^{p}$

but that is still in L

try $xy^{2}z = 0^{p+k} 1^{p} \notin L$

this contr 3 so L is not

regular

Problem 1

Show that the following languages are not regular:

(a) $\{0^i 1^j : 0 \leq i \leq j\}$ $=: L$

Prove by MN Thm

Suppose L is regular

Define $u_j = o^i$

Want to show that

for $i < j$

that there is a w_{ij}

such that

$o^i w_{ij} \in L$ and $o^j w_{ij} \notin L$

$w_{ij} = 1^i \quad (= u_i w_{ij})$

then $0^i 1^i \in L$

but

$u_j w_{ij} = 0^j 1^i \notin L$

since $j > i$

Problem 1

Show that the following languages are not regular:

(b) $\{w \in \{0,1\}^* :$ the number of 0's in w is equal to the number of 1's in $w\}$

Problem 1

Show that the following languages are not regular:

(c) $\{ww : w \in \{0,1\}^*\}$

W3 P2

Problem 2

Give CFGs that generate the following languages:

(a) $\{0^i 1^j : 0 \leq i \leq j\}$

$S \to 0S1 \mid A$

$A \to \varepsilon \mid 1A$

Call G

$N = \{S, A\}$

$\Sigma = \{0, 1\}$

For istance $0^2 1^3 \in L$

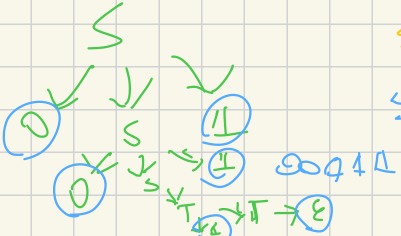
00111 want to produce from G

$S \Rightarrow_G 0S1 \Rightarrow_G 0(0S1)1$

$= 00S11$

$\Rightarrow_G 00A11$

$\Rightarrow_G 00A111 \Rightarrow_G 00111$



Problem 2

Give CFGs that generate the following languages:

(b) $\{w : w \text{ contains } 00\}$

$S \to T00T$

$T \to 0T \mid 1T \mid \varepsilon$

Problem 2

Give CFGs that generate the following languages:

(c) $\{0^i 1^j 0^j 1^i : i, j \geq 0\}$

$S \to 0S1 \mid T$

$T \to 1T0 \mid \varepsilon$

W3 P3

Problem 3 (Part 1)

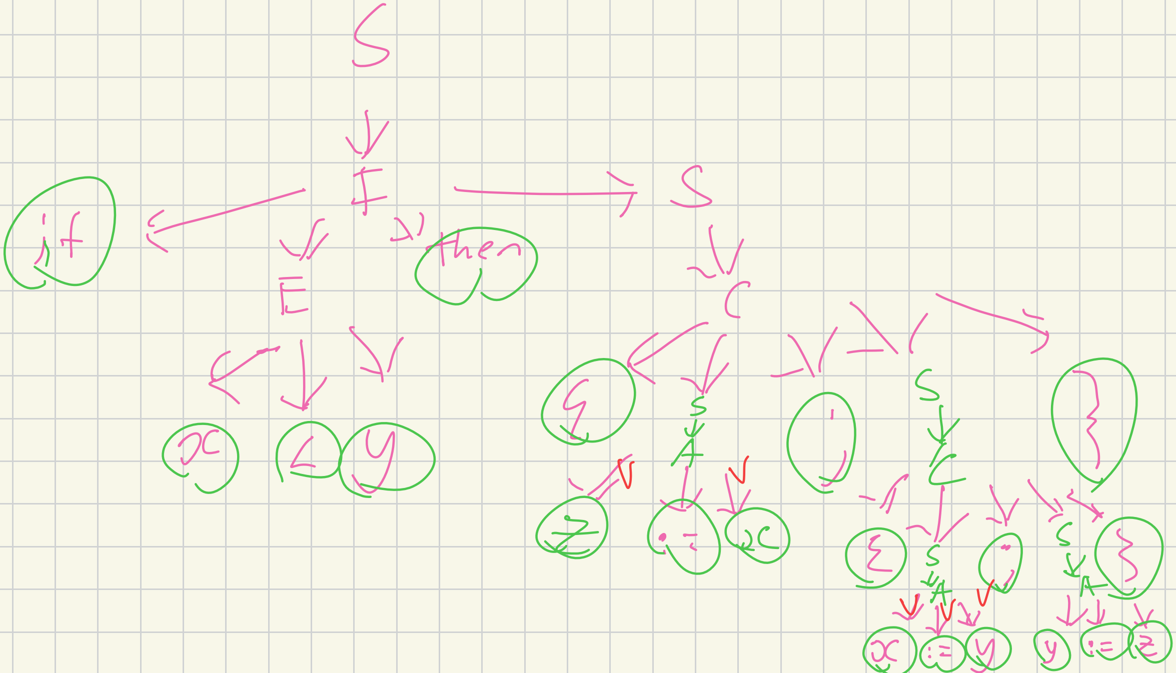
Consider the following context free grammar for a simple programming language:
$G = (N, \Sigma, P, S)$, where $N = \{S, I, A, C, E, V\}$, $\Sigma = \{x,\ y,\ z,\ <,\ :=,\ ;,\ \{,\ \},\ \textbf{if},\ \textbf{then},\ \textbf{else}\}$.
Here the inner braces are terminal symbols and the outer braces are part of the set notation. Treat the keywords **if**, **then**, **else** as if each is a single terminal symbol rather than a sequence of letters.

Problem 3 (Part 2)

(a) Show the parse tree for a simple program in $L(G)$ that sorts the variables x, y into ascending order.

Here P contains the following productions:

$$
\begin{aligned}
S &\to I \mid A \mid C \\
I &\to \textbf{if } E \textbf{ then } S \\
I &\to \textbf{if } E \textbf{ then } S \textbf{ else } S \\
A &\to V := V \\
C &\to \{S; S\} \\
E &\to V < V \\
V &\to x \mid y \mid z
\end{aligned}
$$


S
I
if
E
then
S
x
<
y
C
{
S
;
S
}
A
z
:=
x
C
{
S
;
S
}
A
A
x
:=
y
y
:=
z

a) Program is

if $x < y$ then $\{ z := x; \{ x := y; y := x \} \}$

Problem 3 (Part 2)

(b) Show that this grammar is ambiguous, by showing the parse trees for a string in the language that has two distinct parse trees. (Hint: consider nested uses of the I productions.)

Here P contains the following productions:

$$S \longrightarrow I \mid A \mid C$$

$$I \longrightarrow \textbf{if } E \textbf{ then } S$$

$$I \longrightarrow \textbf{if } E \textbf{ then } S \textbf{ else } S$$

$$A \longrightarrow V := V$$

$$C \longrightarrow \{S; S\}$$

$$E \longrightarrow V < V$$

$$V \longrightarrow x \mid y \mid z$$

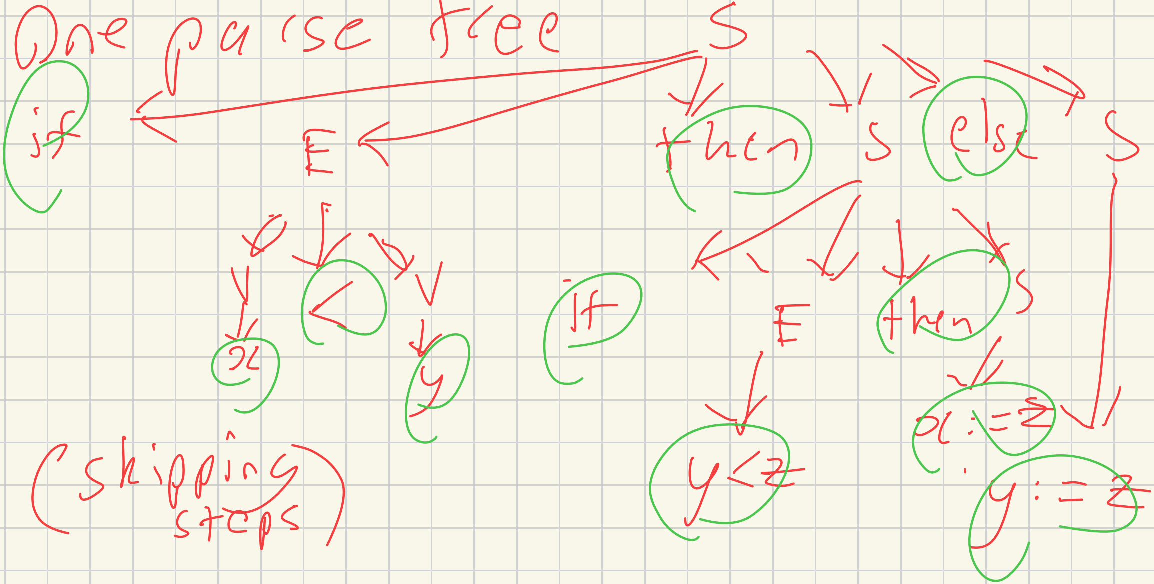
if E then (if E then S) else S

if E then (if E then S else S

Choose

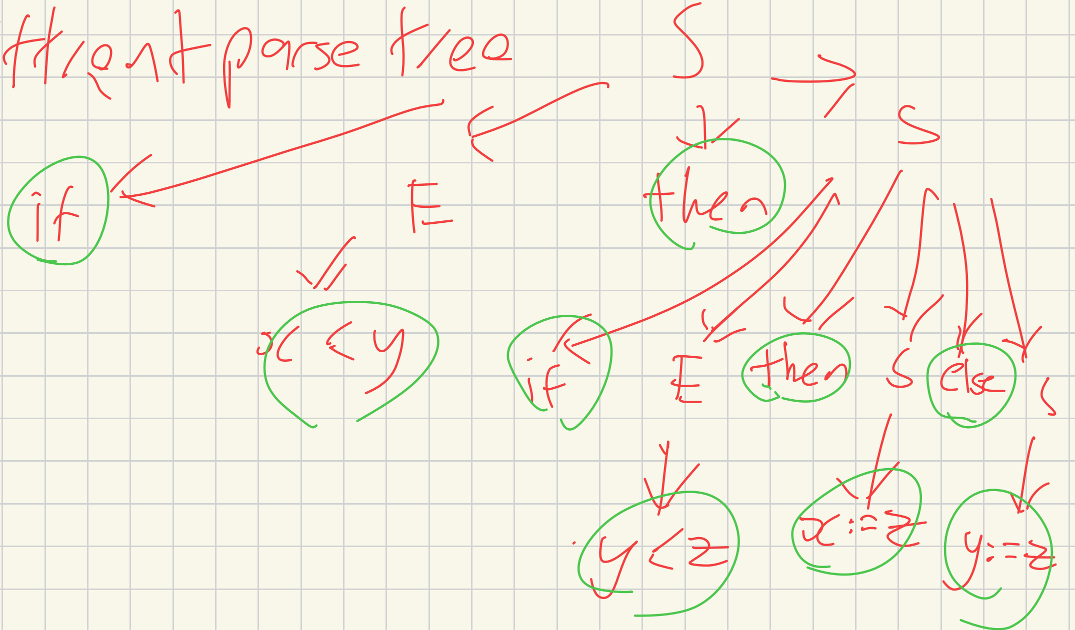
if x < y then if y < z then x := z else y := z

One parse tree



(skipping steps)

Different parse tree

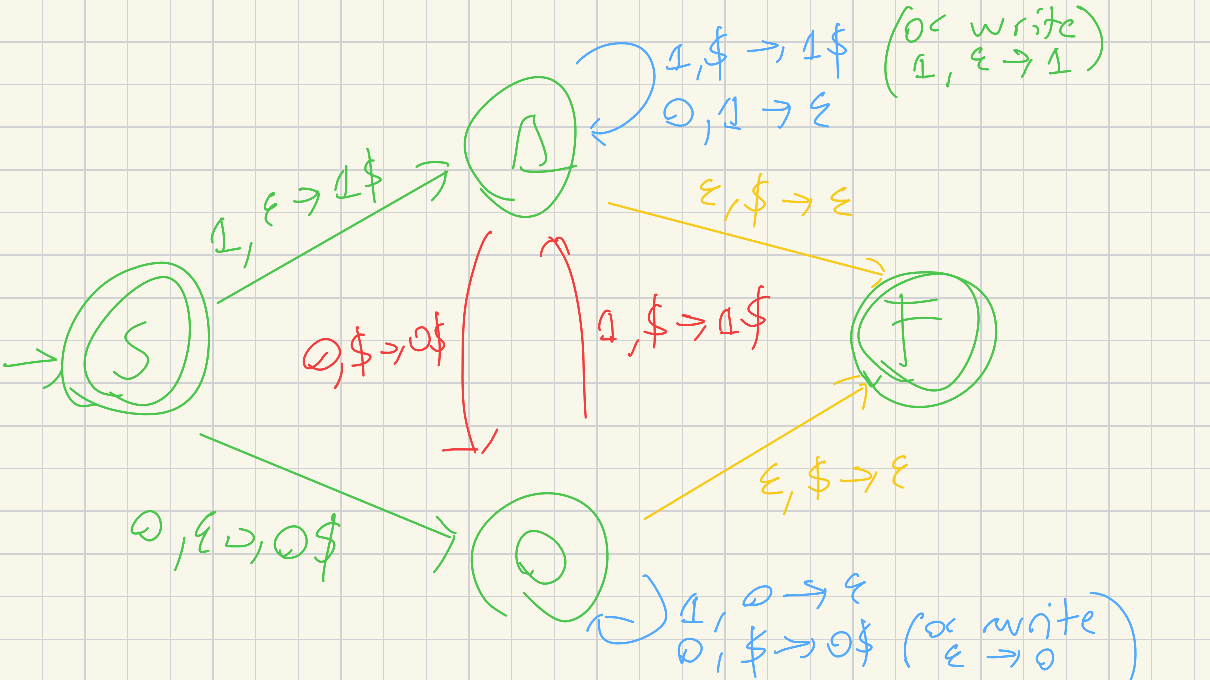


W3 P4

Problem 4

Give PDAs that recognise the following languages:

(a) $\{w \in \{0, 1\}^* :$ the number of 0's in w is equal to the number of 1's in $w\}$


S
1
0
F
1, ε → 1$
0, ε → 0$
1, $ → 1$
0, 1 → ε
(or write
1, ε → 1)
ε, $ → ε
0, $ → 0$
1, $ → 1$
ε, $ → ε
1, 0 → ε
0, $ → 0$ (or write
ε → 0)

Problem 4

Give PDAs that recognise the following languages:

(b) $\{0^i 1^j 0^j 1^i : i, j \geq 0\}$

Problem 4

Give PDAs that recognise the following languages:

(c) $\{w \in \{0, 1\}^* :$ the number of 0's in w is twice the number of 1's in $w\}$